

Agentic AI Summit · 企业级 Agent 开发工具链

从 BizDevOps 到 VibOps

构建 AI 时代从需求到交付的完整闭环

个人提效 → 组织交付力

况雨平 (Leo)

目录

CONTENT

01 生产力之变

AI 让节点更快，组织交付为何没更快？

02 生产关系之困

节点提速之后，研发协作关系如何适配？

03 需求闭环之实

VibOps 如何让一个需求更快、更好地完成？

04 交付进阶之路

我们已经怎么用，下一步走向哪里？

01

生产力之变

AI 让节点更快，组织交付为何没更快？

AI 让节点更快，交付为什么没有等比提升？

01 真实观察：节点确实变快了

内部观察 | 2026-01 — 2026-03 | 50+ 研发

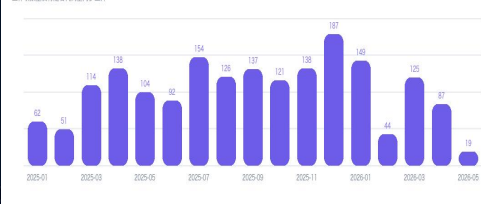
部分编码环节产出效率约 **2-4×**

仅描述部分编码环节，不代表端到端需求周期或组织吞吐



上线需求变化

工作流数量与阶段数代码量同步上升



02 完整交付时钟：一个需求远不止写代码

被验证提速的局部作业段



理解与约束

规则 | 历史 | 架构 | 权限



变更与集成

复用 | 接口 | **【编码】** | 依赖



验证与运行

Review | 测试 | 灰度 | 业务接受

03 机制示意：同一条交付时钟，只压缩其中一段

假设完整交付周期 = 100

提速前

其余环节 80

编码 20

= 100

提速后

其余环节 80

编码 2

= 82

整体缩短约

18%

Amdahl 定律推算 | 假设 Coding 占 20%、该段提速 10×、其余环节不变 | 非内部实测

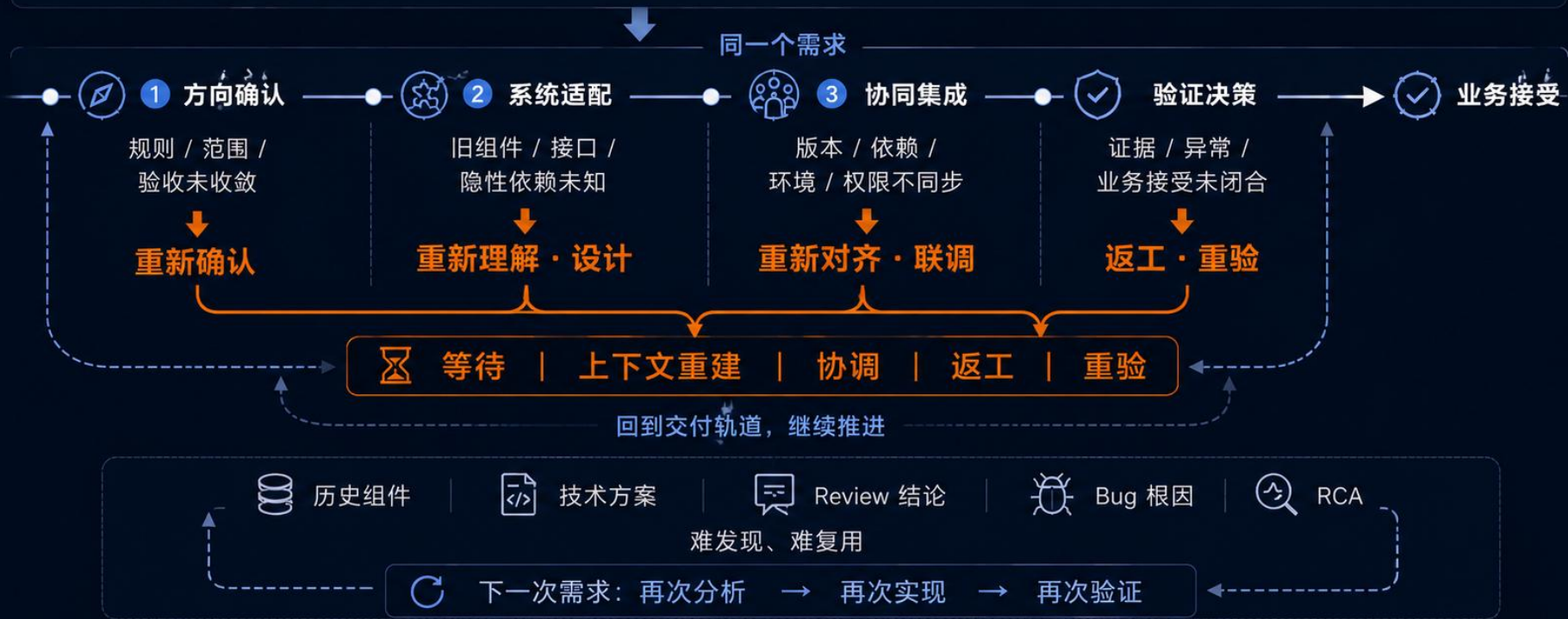


节点作业时间下降，**不等于**端到端交付周期等比下降。

节点节省下来的时间，被什么重新消耗了？



AI 更快完成：分析草稿 | 方案草稿 | 代码实现 | 测试生成
节点内部实际作业时间下降



省下的是首次产出时间，消耗掉的是让答案真正可交付的反复。

02

生产关系之困

节点提速之后，研发协作关系如何适配？

流程在传递产物，需求却还要靠人串起来

- 需求：退款列表增加“批量审批”
- 变化：允许部分成功，失败项展示原因并支持导出
- 规则 | 权限 | 旧组件 | API | 调用方 | 错误处理 | 日志 | 测试 | 验收

正式产物轨道
(专业分工与流程)



需求连续性
断点 (间隙)

① 规则未清
怎么算成功?

② 旧组件与接口
还能复用吗?

③ 版本与依赖
现在同一版吗?

④ 环境与验收
证据齐了吗?

⑤ 经验与完成
下次找得到吗?

人的协作轨道
(临时承接连续性)

问清规则
澄清范围、验收
与边界条件

找历史 Owner
查历史方案、组件、
接口与使用经验

拉群协同
同步版本、依赖、
环境与权限

协调验证
补齐证据、复测、
联调与冒烟

判断完成与复用
确认上线、沉淀经验、
便于下次复用

返回到上轨，推动流程继续前进

不同的人，在不同时间，用沟通、记忆和判断维持同一个需求



流程传递了专业产物，但同一个需求的连续性，仍然主要由人维持。

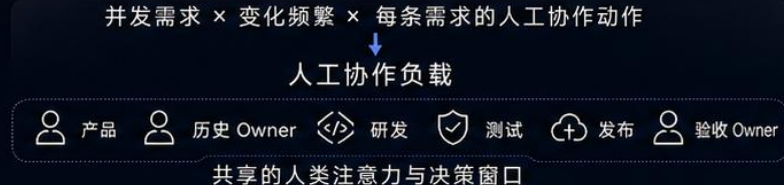
AI 放大了产出并发，需求连续性不能再只靠人的注意力

每条需求都要重复人工协作动作 → AI 带来更多产出、并发与变化 → 多条协作轨道竞争同一组人的注意力 →
等待、切换、漂移和重复集中出现 → 系统承接连续性，人保留判断与责任

第一层 | 从 1 条需求，
放大到 N 条并发需求



第二层 | 人工协作运行时：
分布在多人之间的共享队列



共享队列的可观察结果



第三层 | 责任迁移：
系统承接连续性，
人保留判断与责任



人不是瓶颈；需求连续性只能依赖人的注意力，才是组织扩展的上限。

共同约束 × 个体化 AI

要求协作围绕同一需求闭环

共同面对的企业约束

业务合规
规则·合规·口径

工程历史
架构·依赖·技术栈

运行交付
权限·发布·审计



个体化的 AI 使用

能力不同

输入不同

规范知识未连接

经验产出未沉淀

同一需求 → 不同理解 / 不同方案 / 不同安全边界

组织级共同机制

同一需求对象

同一上下文

同一责任

同一完成标准

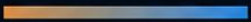


不是让每个人独自把 AI 用得更好
而是围绕**同一需求对象**，统一上下文、责任和完成标准

03

需求闭环之实

VibOps 如何让一个需求更快、更好地完成？



VibOps 如何适配 AI 生产力提升

能力分层与智能交付闭环

AI 提升执行供给, VibOps 同步重构上下文、协作、执行、验证与知识回流



i VibOps 定义协同范式, spec-first 负责 AI Coding Harness, 姬蕾负责统一入口与执行, 知识 / 规范 / 记忆 / Evidence 构成可持续进化的上下文底座。

不同角色进展，随时重连进度

创建需求，就是创建一个可运行的 Workspace

用同一个 work_id 连接需求工件、代码仓库与运行状态



Workspace 不是资料目录，而是同一需求的版本化共同现场和运行边界。

```
workspace/
├── workspace.yaml # Work Object 元数据、状态和关联关系
├── 需求/
│   ├── 原始需求.md # 产品最初输入，允许模糊
│   ├── 澄清问题.md # 待确认问题、答案、决策人与时间
│   ├── PRD.md # 澄清后的详细需求
│   ├── 业务规则.md # 规则、条件、状态和计算口径
│   ├── 验收标准.md # 可验证的业务完成标准
│   ├── 非目标.md # 明确本次不做什么
│   └── 需求变更记录.md # 需求变更原因、影响和确认结果
├── 方案/
│   ├── 技术方案.md # 总体实现方案和关键技术决策
│   ├── API Contract/
│   │   ├── openapi.yaml # 可机器读取的接口契约，按项目实际选用
│   │   └── 接口说明.md # 接口语义、错误码和兼容说明
│   ├── 影响分析.md # 受影响系统、模块、数据和历史功能
│   ├── 风险与回滚.md # 风险、灰度、回滚和停止条件
│   └── 开发任务/
│       ├── TASK-001.md # 可执行的开发任务
│       ├── TASK-002.md
│       └── TASK-003.md
├── 设计/
│   ├── 原型/
│   │   ├── 页面设计/
│   │   ├── 交互说明.md
│   │   ├── 页面状态.md
│   │   └── 设计变更记录.md # 空态、加载、异常、权限等状态
│   └── ...
├── 测试用例/
│   ├── 功能用例.md
│   ├── 异常场景.md
│   ├── 边界场景.md
│   ├── 权限用例.md
│   ├── 接口测试.md
│   ├── 回归用例.md
│   └── 测试数据说明.md
├── solutions/
│   ├── bugfix/
│   ├── code-review/
│   ├── testing/
│   └── delivery/ # 当前需求执行中形成的候选经验
│   # Bug 定位、修复和防复发经验
│   # Code Review 问题与改进方式
│   # 测试遗漏、异常和回归经验
│   # 配置、发布、日志和交付经验
├── 复盘/
│   ├── 复盘报告.md # 目标、决策、变更、Evidence 与过程结论
│   └── 改进项.md # 改进事项、Owner、状态和关联工件
├── repos/
│   ├── 项目1/ # 独立代码 Git 工程
│   ├── 项目2/ # 独立代码 Git 工程
│   └── ...
└── .gitignore # 排除 repos 下的独立代码工程
```

AI 不缺资料和工具，缺的是这次需求的正确装配

按需组合知识、代码、规范、模板、Skill、Agent 与 MCP

DataFun.

AGENTIC AI
超级智能体系架构峰会

概览 文档详情 附件 成员 配置 发布单 代码仓库 活动

❗ 项目关联的知识库与 AI 组件（Skills / Rules / MCP）。这些配置会注入到成员下载的 IDE 配置中。

关联知识库

irs产品方案 (Global) 会议纪要

AI 组件

Skills 文档提交提醒 代码仓库检索助手 global 编写skill 需 档编写助手 Code Review 助手 技术方案评审 requirement-clarification 接口设计助手 测试用例生成

Rules Git 提交规范 Python 编码规范 SQL 编写规范 Vue3 前端规范

MCP VibOps 知识库 MCP hk_application_knowledge_base

概览 文档详情 附件 成员 配置 发布单 代码仓库 活动

关联历史代码仓库作为 AI 知识来源（仅存储在服务端知识库，供 MCP 检索）。平台每隔一段时间会自动同步一次。

全部同步

+ 添加应用

+ 添加仓库

ar_base i-gl-s cription master 手动添加 已同步

60 个文档 · 最后同步: 2026-07-24T17:57 · 6fe1b430

同步

编辑

全量

删除

代码-codegraph 精准辅助

hs i-_/hs-s _dp-admin-api master 手动添加 已同步

91 个文档 · 最后同步: 2026-07-24T17:57 · 0f6df289

同步

编辑

全量

删除

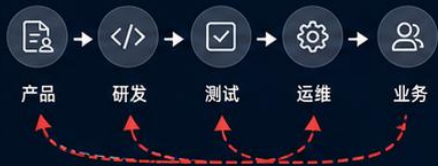
需求级 Workspace：人和 Agent 如何进入同一需求

从专业节点水平接力，转向围绕功能结果的垂直闭环

DataFun.

AGENTIC AI
SUMMIT

过去 | 水平接力



等待 | 转译

口径分叉 | 返工

各守节点产物

PRD / 需求 代码 / PR 测试报告 制品包 运行报告

一个需求级 Workspace

AI 工作区

当前项目: CRM-GSC 公司行动查询需求-创建日期-查询条件变更 运行中

文件: 01-规划, 02-开发, 03-测试, 04-部署, 05-运维, 06-业务, 07-设计, 08-数据, 09-安全, 10-其他

直接读代码，生成原型

基于知识库梳理现状，知识库不存在，直接精确索引代码

找到了，GSC 公司行动查询需求的查询接口信息如下：

项目	内容
方法	POST
端点	/corpaction/gsc/customer/page
请求参数类型	ISccCustomerPageReq

与“创建日期”相关的请求参数：

字段	类型	必填	说明
createDayStart	string	否	创建时间起始

请使用 skill 'prd-writing' 来处理以下请求：将上面对话内容，输出一个prd文档。

(2) 对话澄清需求，prd skill 生成需求prd文档

同一 Work Object | 共享上下文与挂载工件 | 状态 · Evidence · Owner

业务决策 | 方案实现 | 测试交付 | 治理责任

现在 | 垂直功能闭环



多角色共同对结果负责



Web | 治理与验收

查看 Workspace、审批 Contract、验收 Evidence



IDE / Agent | 读取与执行

读取 Context / Contract、执行开发 / Review、回写工件与状态



API / Event / MCP | 事实与状态回写

同步工程事实、状态、Evidence 与流程事件

Workspace 统一需求语义，但不复制工程事实、不成为第二真相源。

从水平接力到垂直闭环：一个功能在同一 Workspace 内完整交付。

企业知识如何从 0 到 1 建立，并在交付中持续进化

代码知识链建基座，交付知识链长增量

身份锚点

work_id

run_id

contract_version

代码知识链 | 0→1 建基座



让代码被理解，让复杂变可导航

交付知识链 | 1→N 长增量



让交付沉淀为资产，让经验持续复用

VibOps 知识控制面

ID / Owner | 来源 / 版本 / 血缘 | 权限 | 生命周期 | 质量 | 可信 | 证据 | AI 能力

K1

K2

K3

K4

全局组装公式 = 内容 + 归属 + 来源 + 约束 + 执行件 + 记忆 + 证据

本次 Context Bundle

工程基座

业务约束

可执行资产

组织记忆

下一页 |

Context Snapshot

未来 Evidence / RCA / 复用反馈回到治理阀门，驱动知识持续进化

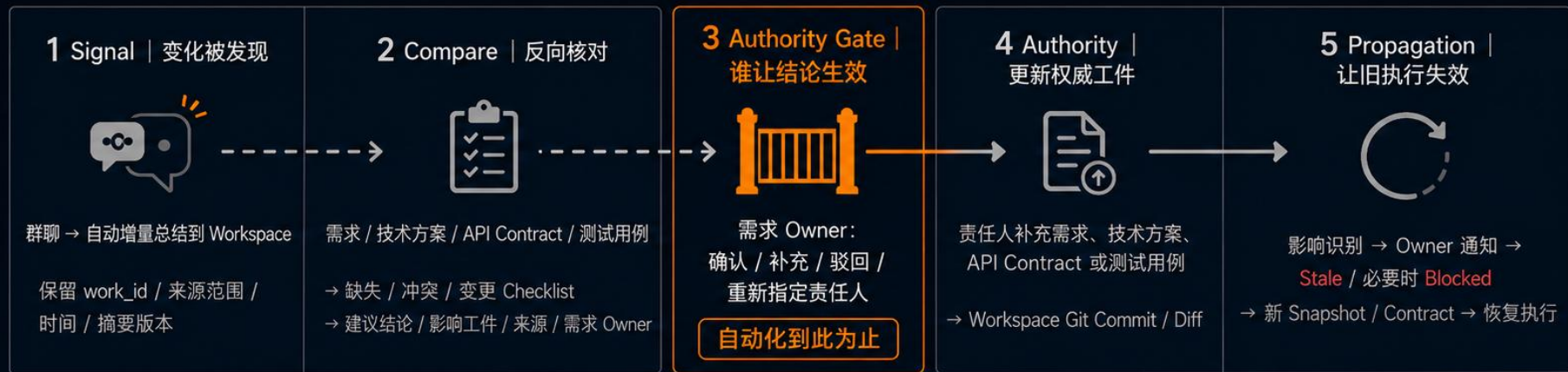
两条知识链让企业知识从可检索走向可执行、可验证、可进化

统一窗口，不是收消息，而是让变化生效

用同一个 work_id 让群聊变化进入权威工件，再触发失效与重算

群聊变化信号 → 自动增量总结到 Workspace → 反向核对并生成 Checklist → 需求 Owner 逐项确认 → 更新权威工件并 Commit → Stale / Blocked 与 Snapshot 重算

群聊贯穿全生命周期：需求澄清 → 方案评审 → 研发 / Review → 测试 / Bug → 发布 → 归档



自动化边界：未确认内容不进入 Snapshot / State / Archive

统一窗口不是把消息收进来，而是让影响交付的变化最终改动同一版权威事实。

Context Snapshot + Contract: 冻结事实与行动边界

DataFun.

AGENTIC AI
SUMMIT

需求承诺点 | work_id=REQ-VIBOPS-017 · run_id=RUN-2026-001 · contract_version=v1

work_id

REQ-VIBOPS-017

run_id

RUN-2026-001

contract_version

v1



Context Snapshot (事实底座)

v2026.05.26-1015

生成时间: 2026-05-26 10:15

基于哪些事实工作 (事实)

- 业务目标 / 港股IPO需求
- 项目 / 代码 / 依赖版本
- 引用知识 / 规范 / 历史决策 / Evidence + 来源
- 装配身份 / 风险等级 (作为事实)
- context_snapshot_version
- 来源系统: VibOps / GitLab / K8s / 知识库



在冻结事实之上划定边界 (向下承接)



Contract (行动与完成边界)

v1



Feature Owner
业务结果



技术 Owner
变更安全



验收 Owner
Evidence / 接受

✓ Contract Approved

三 Owner 批准即执行起点 (Contract Approved)

允许做什么 / 怎样算完成 / 何时必须停

- 目标 / 范围 / 非目标 / 验收示例
- 授予权限 (边界) / 允许工具 (边界)
- 验收标准 / Evidence 要求 / 完成定义
- 资源预算 (边界) / 风险处置边界 / 停止条件 (边界)



★ Required Evidence

需产出与归档的证据清单



★ 停止条件

何时必须停止与阻断



★ 三 Owner

Feature / 技术 / 验收

版本规则 (蓝)



work_id 不变
始终指向同一需求身份



新 run_id
每次执行生成新的 run_id



Contract 变才发新
Contract 变化时才发新
contract_version

阻断规则 (红)



未源冲突 (事实不一致)



事实过期 (Snapshot 过期)



权限不全 (必需未授予)



验收定不下 (标准/Owner 不满足)

→ Blocked, 不补造



执行前先冻结“基于哪些事实 · 允许做什么 · 怎样算完成 · 谁负责”——
AI 可控、可追责、可复现的起点，也是质量与决策前移的落点。

高手的方法，不能只留在高手手里

让正常交付与 Debug 遵守同一套可验证协议

DataFun.

AGENTIC AI
超级智能体系架构峰会

统一工程协议：



正常交付主链：



Failure / Bug
不靠猜：



知识晋升：



两类 Git 的边界：



Workspace Git :

版本化需求、方案、测试与经验工件



代码仓库 Git :

保存真实源码、Commit、分支与 PR 真相

 **Workspace 不是资料目录，而是同一需求的版本化共同现场和运行边界。**

VibOps 如何低成本接入现有研发平台？

MCP 将已有能力工具化，妮蔻脚本负责确定性执行

DataFun.

AGENTIC AI
超级智能体系架构峰会



低成本点题：不替换、不迁移、不重建平台，只增加受控调用与结果回写。

Evidence + Owner Acceptance: 什么才算完成

Agent completed ≠ 完成

Test Gate passed ≠ 完成

Deployed ≠ 完成

Business Accepted = 需求完成

双阶段决策主链

阶段 1

发布前



Evidence 采集

功能实现 / 代码 /
单测 / 集成 / 安全等



Bundle / Gate

证据打包 / 门禁校验
(质量 / 风险 / 覆盖等)



**Human
Release Approval**

人工发布审批



Deploy

发布到目标环境

阶段 2

部署后



Evidence 采集

运行验证 / 监控 /
日志 / 性能 / 数据等



Acceptance Gate

验收门禁
(业务 / 体验 / 指标等)



**Owner
Acceptance**

Owner 验收确认



Business Accepted

需求完成

驳回 / 补证 / Rework / Rollback



判定与责任



Record

所有关键 Evidence 留痕
可追溯 / 可审计



Bundle

上下文证据包
(本次交付 完整证据集)



Gate Verdict

门禁判定结果
通过 / 驳回 / 需补证



Owner Decision

Owner 业务接受
负责结果与价值



spec-first 规定如何工作

· **Evidence** 证明做成什么

· **Owner** 决定是否接受

让每一次交付，都成为下一次需求的起点

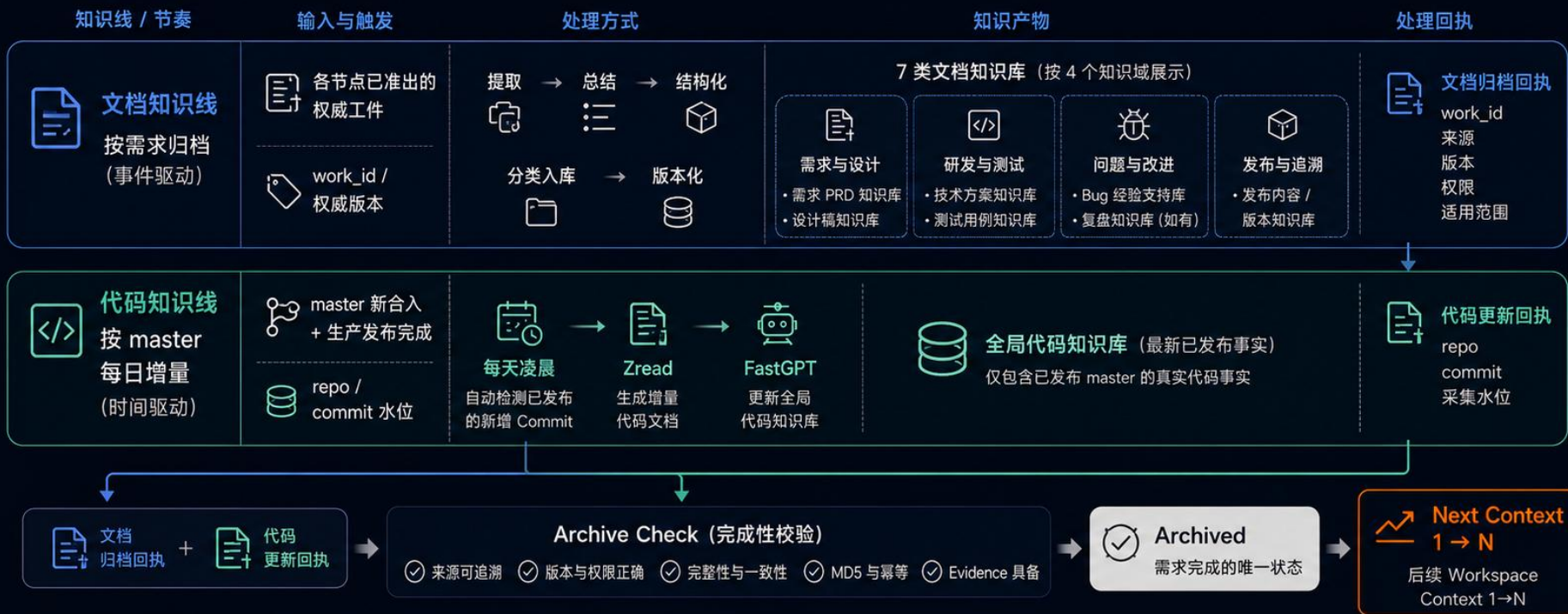
文档随需求归档，代码随 master 每日增量更新

DataFun.

AGENTIC AI
超级智能体系架构峰会

Archive Running | 文档归档 + 代码更新 → 两类回执

两类回执共同通过 Archive Check，需求才进入 Archived



一次需求沉淀文档判断，一次发布沉淀最新代码；两条线共同抬高下一次需求的起点。

04

交付进阶之路

我们已经怎么用、下一步走向哪里？

产研测共享工作空间，已进入 App 与 Admin 真实交付

DataFun.

AGENTIC AI
超级智能体系架构峰会

每个需求一个 Workspace

产研测共享上下文 · 同一需求事实 · 同一完成标准

进入下一个 Workspace

APP | 单人双端研发流程

共享需求语义 → Android + iOS → 双端验证

7 个已完成需求 | 效率 +45% | Bug -80%+

ADMIN | 后端跨栈研发流程

API Contract → 服务 → Admin 页面 → 测试

Admin 开发已全部由后端研发承担

知识增长飞轮 | 意图识别 → 精准回捞 → 当前交付 → 知识归档与治理 → 下一需求 ↻

下一步：让每一次交付，持续放大生产力

DataFun.

AGENTIC AI
超级智能体系架构峰会

覆盖更多需求 | 承担更多任务 | 兑现更多组织价值

✓ 已完成起点 | 需求闭环 + 知识复利已在真实需求中运行

覆盖更多

01 意图识别 + 精准回捞

知识治理 → 任务 / 角色 / 阶段
正确版本 / 权限 / 相关性



承担更多

02 任务看板 + 多 Agent 协同

同一 Work Object
依赖 / WIP / Owner / Gate / Evidence
人机协作 → 受控自治

兑现更多

03 数据看板反哺流程优化

E2E / 等待 / 返工 / 质量 / 知识使用 / 人工接管
→ 知识治理 / 流程 / Context / Rule / Skill /
Test / 模型路由

系统基于 Evidence 持续进化

数据反馈回流到知识治理、流程编排、Context 与能力资产



token 成本治理 | 贯穿全链路

单需求 / 单任务 | 无效上下文 | 有效检索与生成 | 模型路由

压缩无效上下文 + 优化模型路由 → 降低单位有效交付成本

人机协作 → 受控自治 | 组织级价值仍在连续验证

每次交付反哺下一次 | 系统越用越准、协同越跑越稳

$$\text{组织交付力} = \text{AI 生产力} \times \text{生产关系适配度}$$

↑
VibOps

AI 让节点更快，VibOps 让端到端流动跟上。

THANKS

演讲嘉宾：况雨平 (leo)



个人微信



spec-first